

Strategic and Technical Measures for a Robust Database Security: A Hybrid RBAC–ABAC Model

Alan B. Palayil
University of the Cumberland
ITS 8310 – Emerging Threats & Countermeasures
Dr. Michael Vubinski
September 28th, 2025

Introduction

Databases function as the primary repositories for an organization's essential assets, including customer records, intellectual property, operational telemetry, and regulated information such as PII, PHI, and PCI. Modern attackers increasingly target data stores directly, necessitating more than ad hoc hardening for database security. A systematic approach that combines strategic governance with layered technical safeguards is essential throughout the entire lifecycle, from design to decommissioning.

Organizational data is frequently referred to as the "crown jewels" due to its critical role in decision-making, fostering customer trust, and ensuring regulatory compliance. Databases are particularly vulnerable to various threats, including SQL injection, privilege escalation, ransomware, and insider misuse. For protection to work, it must encompass both strategic governance (policy, risk management, compliance alignment, and defined security responsibilities) and technological controls (hardening, encryption, monitoring, and recovery methods).

This paper outlines strategic and technical controls for robust database security, develops a hybrid defense-in-depth model that integrates Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), and cryptographic mechanisms, and explains the design, operation, and decommissioning of databases within this framework. The proposed model integrates integrity assurances derived from the Clark-Wilson architecture and utilizes recognized standards, notably NIST SP 800-53 Rev. 5. This talk is based on foundational research on access control models (Penelova, 2021), RBAC implementations (Figuerola-Lorenzo et al., 2019), and general database security concepts (Bertino & Sandhu, 2005).

Strategic Security Measures (Program, Governance & Culture)

1. **Organizing and controlling data:** Organizations need to sort data into groups based on how sensitive it is (e.g., Public, Internal, Confidential, Restricted) and then link those groups to rules for managing the data that say where it may be stored, who can access it, what encryption standards to use, how long to keep it, and how to disguise it. Classification is what makes least privilege and data-centric controls function. It helps people follow rules like NIST SP 800-53 Rev. 5 (PM, MP, SC families) and makes governance easier to scale (Bertino & Sandhu, 2005).
2. **Managing Risks and Making Threat Models:** For applications and database management systems (DBMS), systematic threat modeling must be done. This includes misuse situations like SQL injection, authentication bypass, privilege escalation through service accounts, and ransomware assaults. According to recognized standards like NIST SP 800-53 Rev. 5, risks must be matched with mitigations. This includes controls for access, auditing, incident response, contingency planning, and key management. Secure configuration baselines for

databases, risk-ranked results, and service level agreements for remediation all make things stricter.

3. **Change control and a safe software development life cycle:** Security should be a part of every step of the software development process. Before they are put into production, database schemas, stored procedures, and ORM queries should be managed as code and go through code review, static analysis for injection vulnerabilities, and testing with masked data. Any changes to the schema, the addition of new stored procedures, or any privilege escalations must be clearly approved by security before they can be made to Infrastructure-as-Code (IaC) and migration pipelines.
4. **Least Privilege, Privileged Access, and Separation of Duties:** It is important to set up duties so that one person cannot start, authorize, and make modifications that might be bad. Separation of responsibilities (SoD) helps stop fraud and mistakes by following the standards of RBAC (Ferraiolo & Kuhn, 1992) and Clark-Wilson integrity. Privileged access management (PAM) makes control better by letting administrators temporarily raise their access levels for purposes like session recording and checking in and out credentials. You should only use break-glass accounts in emergencies and only after getting written permission (Penelova, 2021).
5. **Plans for keeping things operating, backing them up, and getting back on track after a disaster:** You should encrypt your backups, limit who may see them, and check them often. Business continuity planning includes setting RPO and RTO targets, geo-redundant replication, and keeping snapshot services. It is important to assess restoration procedures on a regular basis to make sure that systems can still recover from ransomware attacks and other issues.
6. **Rules for Software as a Service and Vendors:** If you use managed databases or SaaS solutions, you should talk to your vendors to find out what kind of security guarantees they offer. These guarantees should include things like tenant isolation, cryptographic modules that have been checked by FIPS 140-3, customer-managed encryption keys, audit logs, and choices for where to store data. This monitoring is necessary to make sure that internal controls and outsourced infrastructure work well together.
7. **A culture of safety and being ready to work:** In a culture that values safety, technical measures work better. Each worker needs training that is tailored to their job. It is important for developers to learn about ORM and parameterized queries, for DBAs to learn how to manage encryption keys and audits, and for analysts to learn how to manage sensitive data extracts. Being aware of the workforce makes it easier to put policies into action, which leads to better government.
8. **Following the Rules:** All actions must follow rules and laws, such as HIPAA, PCI DSS, GLBA, SOX, and GDPR. Having data ready for an audit is important to make sure that everyone is responsible and follows the rules throughout evaluations. This includes keeping an eye on who has access, asking for permission to make changes, keeping audit records, and evaluating several ways to recover.

Technical Security Measures (Defense in Depth)

1. **Managing Identity and Access:** RBAC as a starting point and ABAC for fine-tuning: Use role-based access control (RBAC) to give people rights based on their job duties (e.g., AR_Analyst, DataScientist_ReadOnly, DB_Ops_Backup), not on their names. Attribute-based access control (ABAC) adds further detail by using factors including data classification, time, device posture, and risk score (Ferraiolo & Kuhn, 1992; Penelova, 2021). Least privilege means just allowing the least number of CRUD activities and not allowing SELECT * queries.
 - **Advanced controls:** Use row/column-level security (RLS/CLS), dynamic data masking, and view-based access to limit who can see data.
 - **Authentication:** Require MFA for service principals and admins, use short-lived tokens, and use mutual TLS for communication across services.
 - **Access mediation:** bastion hosts, Zero Trust Network Access (ZTNA), or identity providers should be used to give those with exclusive access.
2. **Isolate the network and the computer:** Put databases on private subnets or VPCs with security groups that block access by default and host firewalls. Limit outgoing traffic to services that are on the allow list (such KMS/HSM and backup vaults). To limit access to the console, use private endpoints, a VPN, or ZTNA. Harden hosts by using minimum OS packages, tweaking the kernel, setting CIS/STIG baselines, EDR, and regular patching. For workloads that are containerized or virtualized, use SELinux or AppArmor, provide users with the least amount of access they need, and make images as secure as possible.
3. **Securing databases and writing secure code:** Change the settings, turn off things you do not need, and limit hazardous functions (like xp_cmdshell). The best long-term way to protect against SQL injection is to use parameterized queries or SQL produced by an ORM (Halfond, Viegas, & Orso, 2009). Use stored processes for important transactions (Clark-Wilson model). Make sure there are separate schemas for applications and administration, and make sure that object ownership and permissions are very stringent. Use query resource limitations like timeouts and result-set limits to lower the danger of exfiltration.
4. **Key Management and Cryptography:** Enforce TLS 1.2+ with contemporary cipher suites for connections between services and databases. When data is at rest, use Transparent Data Encryption (TDE) for general protection and column/field-level encryption or format-preserving encryption for sensitive fields like SSNs and PANs. Key management: Keep everything in KMS/HSM, make sure that different people do different things, and change keys often. To be compliant, use modules that have been certified by FIPS 140-3 and envelop encryption. Think about using tokenization or deterministic encryption for fields that can be searched.
5. **Keeping an eye on things, auditing, and stopping data loss:** Allow native auditing for logins, permits, modifications to schemas, and access to sensitive items. Use Database Activity Monitoring (DAM) to enforce rules and make virtual patching possible.

Send logs to SIEM and use UEBA to find strange trends, such big exports, strange query forms, and access outside of business hours. Put logs in WORM/immutable systems that show if they have been tampered with. Add DLP rules to keep an eye on outgoing traffic for PII, credentials, or large transfers.

6. **Backup, recovery, and managing the lifecycle:** Use the 3-2-1 strategy: keep three copies, two on different media, and one that cannot be changed. Use different keys to encrypt backups than the ones used in production and use the same or stricter access constraints. Check PITR (point-in-time recovery) and replication to secondary areas. Apply data lifecycle management: enforce retention schedules, TTLs, and cryptographic erasure via key destruction. Make sure that sanitized or masked data is utilized for non-production restorations.
7. **Access from a distance and by third parties:** Require VPN or Zero Trust Network Access (ZTNA) with device posture validation. Stop using shared accounts and provide each user or service with its own credentials. For partners, use application-layer proxies with throttles, query allow-lists, and logging. Monitor and revalidate partner compliance continuously.
8. **Operational Hygiene:** Use Privileged Access Workstations (PAW) for DBAs. Maintain break-glass accounts with time-boxed approvals and enhanced monitoring. Continuous vulnerability scanning, patch management, and configuration drift detection. Mask or de-identify sensitive datasets in dev/test environments via synthetic data or dynamic masking.

Hybrid RBAC + ABAC is a viable choice for database security since it has Defense-in-Depth.

Role-Based Access Control (RBAC) is still easy to use and check, and it answers the key question of "who can do what." But Attribute-Based Access Control (ABAC) provides context, allowing for dynamic restrictions like an analyst can only query internal data from a controlled device during business hours, but restricted PII needs an on-call ticket reference. Combining RBAC with ABAC gives you fine-grained enforcement without making too many roles (Penelova, 2021). Research indicates that RBAC is validated in centralized control settings like SCADA (Figuerola-Lorenzo et al., 2019), and its extension with ABAC efficiently scales across diverse corporate contexts.

A model for layered defense-in-depth

The model below shows governance at the top, monitoring and resilience at the bottom, and RBAC/ABAC as the main layer for enforcement. Security controls cover a lot of ground:

- Governance and Policy: guidelines for how to manage data, change control, and compliance mapping.
- Role-based access control (RBAC) and attribute-based access control (ABAC) include role engineering, separation of responsibilities (SoD), multi-factor authentication (MFA), row/column-level security, and attribute rules (time, device, location, label).

- Network and platform controls include segmentation, private endpoints, host hardening, and patching.
- Database Hardening and Secure Coding: Use the principle of least functionality, parameterized queries, and stored procedures to make sure that transactions are well-formed.
- Data Protection & Cryptography: TLS while data is in transit, TDE while data is at rest, field-level encryption (FLE) for sensitive columns, centralized keys in KMS/HSM, masking, and tokenization.
- Monitoring and Resilience: Database activity monitoring (DAM), SIEM/UEBA integration, anomaly detection, immutable logging, DLP, encrypted/immutable backups, and testing for disaster recovery and business continuity.

Hybrid RBAC/ABAC Database Security Model

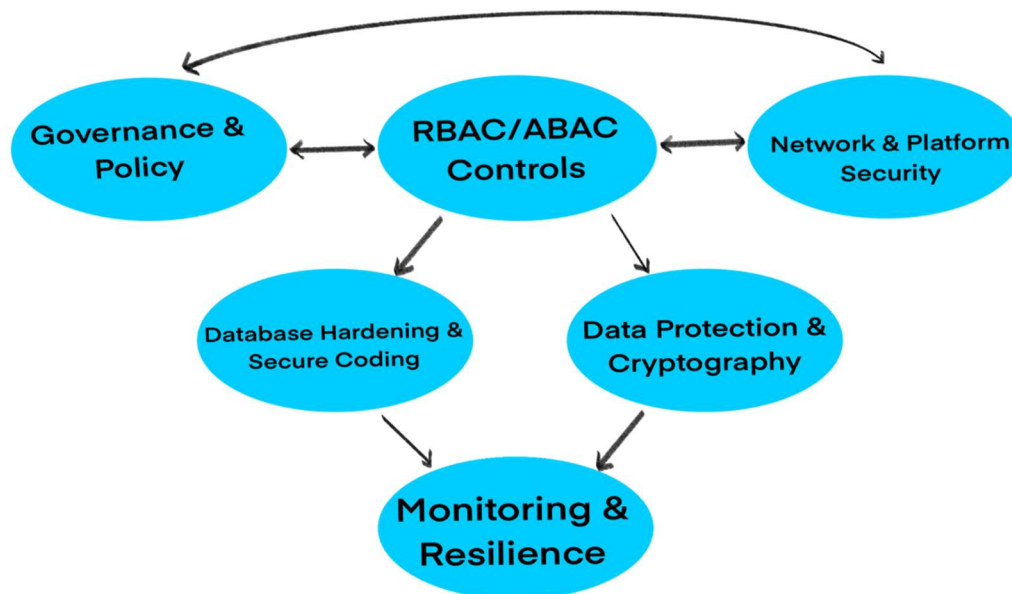


Figure 1. Hybrid RBAC/ABAC database security model.

Clark-Wilson Controls for Integrity

Clark-Wilson model protects how data can be changed with well-formed transactions enforced by stored procedures and application logic. The people who start transactions and the people who certify them have different jobs. Full audit trails for every change. For example, only stored procedures with built-in validation can change accounting tables. Direct table writings are not permitted, and approvals are noted and kept separate from execution. In the end, they said that IoT needs custom solutions like lightweight cryptography, scalable key management, monitoring, and secure updates to stay safe from new threats.

Reducing the risk of common attack paths

This hybrid architecture protects against common database threats:

1. Credential theft and social engineering: MFA, PAM/JIT, and ABAC context checks (which reject unmanaged devices) and DAM warnings make it harder for hackers to take advantage of vulnerabilities (Wang et al., 2020).
2. SQL injection: Parameterized queries, least privilege, and row-level security (RLS) stop privilege escalation. ABAC throttles, statement timeouts, and monitored export allow-lists stop mass exfiltration.
3. Stolen backups: encrypted backups, separate keys, and hidden lower-environment restorations.
4. Insider abuse: logs that cannot be changed, SoD, approval workflows ("break-glass"), and finding strange query patterns.
5. Ransom/extortion: Isolated encrypted backups, evaluated restorations, and escrowed keys make sure you can get your data back.

A practical plan for rollout

1. Tag tables and columns by how sensitive they are (for example, `customer.ssn` → Restricted/PII).
2. Role Engineering: Make sure that business roles match permissions; map users/groups to roles and objects; and put it all in DDL with version control.
3. Attribute Policies: Rules for time, device, and location for sensitive access.
4. Isolation and encryption: Use private endpoints, need TLS, turn on TDE and FLE for PII, and keep keys in one place.
5. Coding Best Practices: Refactor dynamic SQL, make sure that parameters are used, and send writes through verified stored procedures.
6. Auditing and Monitoring: Turn on login, schema, and audit logs; send them to SIEM; and develop SOAR playbooks for occurrences that are very risky.
7. Backup and Resilience: snapshots that cannot be changed and are encrypted; copies stored offshore; recovery exercises every three months.
8. Operations: quarterly recertification of access, rotation of secrets and keys, patch SLAs, and tabletop exercises for incidents.
9. Training and Awareness: Phishing simulations, developer secure-coding refreshers, and hardening the DBA position.

Conclusion

There are many layers of database security, and they all work together to keep your data safe. Good models use hybrid RBAC-ABAC authorization methods that work in complicated situations and

mix role separation, least privilege, and classification. The Clark-Wilson principles protect integrity by making sure that transactions are explicit and that everyone is responsible for their actions. On the other hand, encryption with good key management keeps data private. Data is always there and can be retrieved because of network segmentation, encryption of data in transit and at rest, and constant monitoring with strong backup policies. With this tiered strategy, databases go from being easy targets to strong, well-managed assets. It is based on well-known standards like NIST SP 800-53 Rev. 5 and is guided by access-control theory (Penelova, 2021) and critical-systems practice (Figueroa-Lorenzo et al., 2019). When used in full, it dramatically lowers the risks and effects of credential theft, injection, insider misuse, and data exfiltration. It also gives organizations legal access to the data and produces analytic value that fits the organization's demands for risk and compliance.

References

1. Bertino, E., & Sandhu, R. (2005). Database security—Concepts, approaches, and challenges. *IEEE Transactions on Dependable and Secure Computing*, 2(1), 2–19. <https://doi.org/10.1109/TDSC.2005.9>
2. Ferraiolo, D. F., & Kuhn, D. R. (1992). Role-based access controls. *Proceedings of the 15th NIST–NCSC National Computer Security Conference*, 554–563.
3. Figueroa-Lorenzo, S., Añorga, J., & Arrizabalaga, S. (2019). A role-based access control model in Modbus SCADA systems: A centralized model approach. *Sensors*, 19(20), 4455. <https://doi.org/10.3390/s19204455>
4. Halfond, W. G. J., Viegas, J., & Orso, A. (2009). A classification of SQL-injection attacks and countermeasures. *Proceedings of the IEEE International Symposium on Secure Software Engineering*, 13–15.
5. National Institute of Standards and Technology. (2020). Security and privacy controls for information systems and organizations (NIST SP 800-53 Rev. 5). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-53r5>
6. Penelova, V. (2021). Access control models. *Cybernetics and Information Technologies*, 21(4), 77–104. <https://doi.org/10.2478/cait-2021-0037>
7. Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Communications of the ACM*, 17(7), 388–402. <https://doi.org/10.1145/361011.361067>
8. Wang, P., Sun, L., & Zhu, H. (2020). Defining social engineering in cybersecurity. *IEEE Access*, 8, 85094–85115. <https://doi.org/10.1109/ACCESS.2020.2993676>